Diffusion models

Ole Winther

Department of Biology
University of Copenhagen (KU)

Dept for Applied Mathematics and Computer Science
Technical University of Denmark (DTU)

December 31, 2025

Objectives of this week's lectures

- Variational autoencoder (VAE)
  recap

$$\mathbf{z} \to \mathbf{x}$$

Objectives of this week's lectures

- Variational autoencoder (VAE) recap

$$\mathbf{z} \to \mathbf{x}$$

- Hierarchical VAE

$$\underbrace{\mathbf{z}_0 \to \mathbf{z}_{\Delta t} \to \ldots \to \mathbf{z}_1}_{T+1\,\text{steps}} \to \mathbf{x}$$

Objectives of this week's lectures

- Variational autoencoder (VAE) recap

$$\mathbf{z} \to \mathbf{x}$$

- Hierarchical VAE

$$\underbrace{\mathbf{z}_0 \to \mathbf{z}_{\Delta t} \to \ldots \to \mathbf{z}_1}_{T+1\,\text{steps}} \to \mathbf{x}$$

- From hierarchical VAE to differential equations $\Delta t \to 0$.
  - Diffusion models
  - Score matching
  - Flow matching

Motivation

- GenAI - enough said!
- Auto-regressive

$$p(\mathbf{x}) = \prod_i p(x_i|\mathbf{x}_{<i}) \quad \mathbf{x}_{<i} = x_1, \ldots, x_{i-1}$$

- (Causal) transformers, RNNs

Motivation

- GenAI - enough said!
- Auto-regressive

$$p(\mathbf{x}) = \prod_i p(x_i|\mathbf{x}_{<i}) \quad \mathbf{x}_{<i} = x_1, \ldots, x_{i-1}$$

- (Causal) transformers, RNNs
- Latent variable

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

- Variational autoencoders (VAEs), Generative adversarial networks (GANs) and Normalizing flows, Hierarchical VAEs (this week), Diffusion models (this week)

Motivation

- GenAI - enough said!
- Auto-regressive

$$p(\mathbf{x}) = \prod_i p(x_i|\mathbf{x}_{<i}) \quad \mathbf{x}_{<i} = x_1, \ldots, x_{i-1}$$

- (Causal) transformers, RNNs
- Latent variable

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

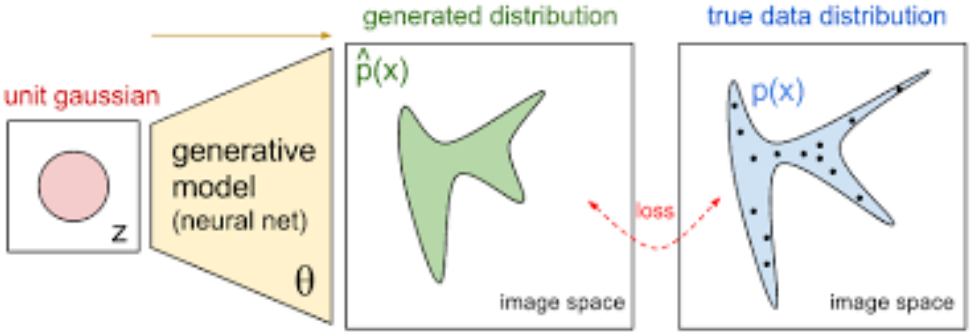
- Variational autoencoders (VAEs), Generative adversarial networks (GANs) and Normalizing flows, Hierarchical VAEs (this week), Diffusion models (this week)

Roadmap

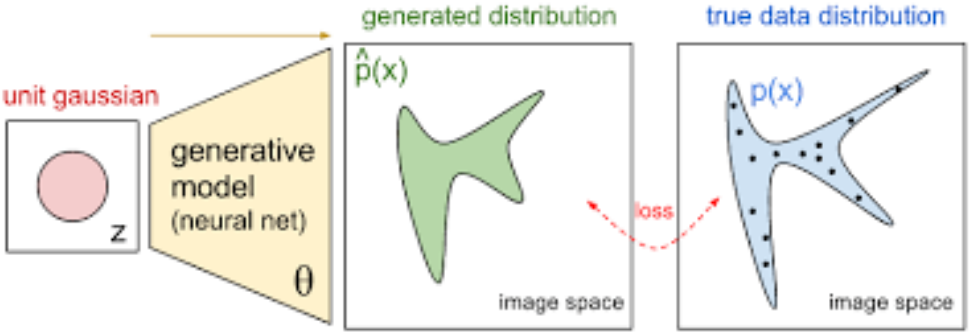
- Recap VAE
- Hierarchical VAE - more layers of $\mathbf{z}$ to get more flexibility/smooth transformations
- Differential equations - the infinite layer limit.
- Popular models using the differential equations: diffusion, score and flow matching

Part 1:
VAE recap

Latent variable model



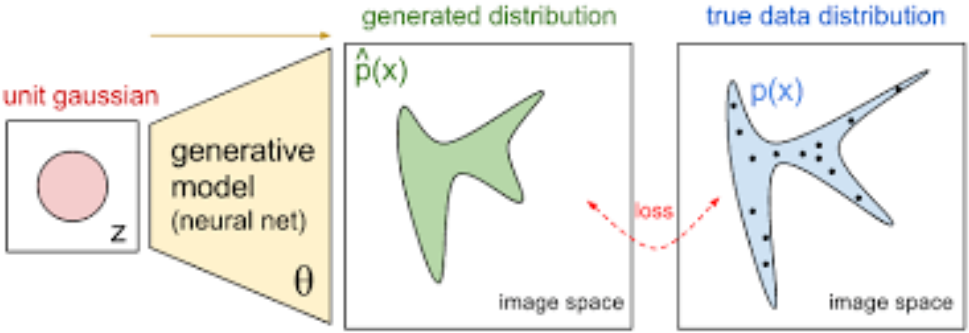
Latent variable model



- Construct distribution $\hat{p}(\mathbf{x})$ from latent $\mathbf{z}$ + transformation:

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

Latent variable model



- Construct distribution $\hat{p}(\mathbf{x})$ from latent $\mathbf{z}$ + transformation:

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

- Marginalisation (treat $\mathbf{z}$ as a nuisance parameter):

$$\hat{p}(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

OpenAI Generative models blog

Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V} \tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

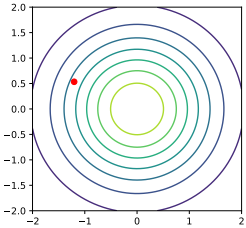
Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)$$

$$\mathbf{z} = (-1.2033, 0.5340)$$



Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V} \tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

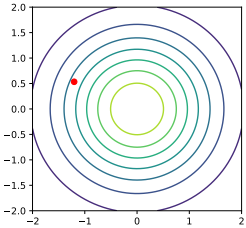
Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)$$

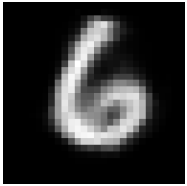
$$\mathbf{z} = (-1.2033, 0.5340)$$



Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

$f(\mathbf{z})$



Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V}\tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}\right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

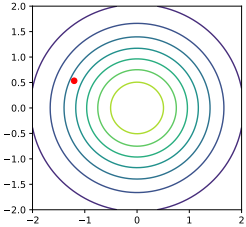
Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V}\tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

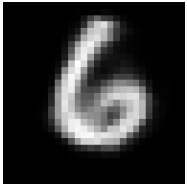
Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}\right)$$

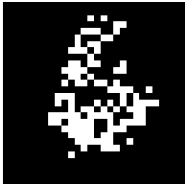
$$\mathbf{z} = (-1.2033, 0.5340)$$



$f(\mathbf{z})$



$x^{j,k} \sim \text{Bern}(f^{j,k}(\mathbf{z}))$



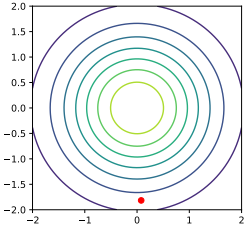
Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)$$

$$\mathbf{z} = (0.0791, -1.8165)$$



Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

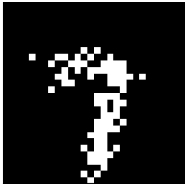
$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V} \tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

$f(z)$

$x^{j,k} \sim \text{Bern}(f^{j,k}(z))$



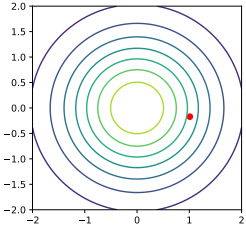
Illustrative example of a (deep) generative model - recap

Training data $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ binary MNIST

Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)$$

$$\mathbf{z} = (1.0097, -0.1711)$$



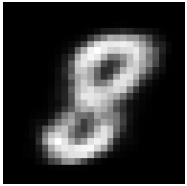
Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0,1\}^{28\times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

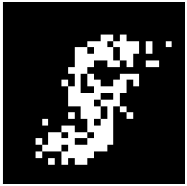
Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V} \tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

$f(z)$



$x^{j,k} \sim \text{Bern}(f^{j,k}(z))$



Latent variable model - easy and hard computationsl

- Latent variable model - joint data and latent (easy)

$$p_\theta(\mathbf{x}, \mathbf{z}) = \underbrace{p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Link+FNN}} \underbrace{p(\mathbf{z})}_{\mathcal{N}(\mathbf{z}|0,\mathbf{I})}$$

Latent variable model - easy and hard computationsl

- Latent variable model - joint data and latent (easy)

$$p_\theta(\mathbf{x}, \mathbf{z}) = \underbrace{p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Link+FNN}} \underbrace{p(\mathbf{z})}_{\mathcal{N}(\mathbf{z}|0,\mathbf{I})}$$

- Sampling also known as inference (easy)

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

Latent variable model - easy and hard computationsl

- Latent variable model - joint data and latent (easy)

$$p_\theta(\mathbf{x}, \mathbf{z}) = \underbrace{p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Link+FNN}} \underbrace{p(\mathbf{z})}_{\mathcal{N}(\mathbf{z}|0,\mathbf{I})}$$

- Sampling also known as inference (easy)

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

- Likelihood - treat $\mathbf{z}$ as so-called nuisance parameter and integrate out (hard)

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

Latent variable model - easy and hard computationsl

- Latent variable model - joint data and latent (easy)

$$p_\theta(\mathbf{x}, \mathbf{z}) = \underbrace{p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Link+FNN}} \underbrace{p(\mathbf{z})}_{\mathcal{N}(\mathbf{z}|0,\mathbf{I})}$$

- Sampling also known as inference (easy)

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

- Likelihood - treat $\mathbf{z}$ as so-called nuisance parameter and integrate out (hard)

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

- Latent posterior from Bayes' theorem (hard to sample from and compute because of $p(\mathbf{x})$)

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$$

Latent variable model - easy and hard computationsl

- Latent variable model - joint data and latent (easy)

$$p_\theta(\mathbf{x}, \mathbf{z}) = \underbrace{p_\theta(\mathbf{x}|\mathbf{z})}_{\text{Link+FNN}} \underbrace{p(\mathbf{z})}_{\mathcal{N}(\mathbf{z}|0,\mathbf{I})}$$

- Sampling also known as inference (easy)

$$\mathbf{z} \sim p(\mathbf{z})$$
$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$

- Likelihood - treat $\mathbf{z}$ as so-called nuisance parameter and integrate out (hard)

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

- Latent posterior from Bayes' theorem (hard to sample from and compute because of $p(\mathbf{x})$)

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$$

- Learning (hard because $p(\mathbf{x})$ hard)

$$\theta_{\text{ML}} = \operatorname*{argmax}_\theta \sum_{j=1}^{n} \log p_\theta(\mathbf{x}^{(j)})$$

The magic of amortized inference!

- Kingma+Welling 2013, Rezende et al 2013:
- Decoder: (example continuous observations) $\mathbf{z} \to \mathbf{x}$

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\underbrace{\boldsymbol{\mu}_\theta(\mathbf{z})}_{\text{FFNN}}, \underbrace{\text{diag}(\boldsymbol{\sigma}^2_\theta(\mathbf{z}))}_{\text{FFNN}})$$

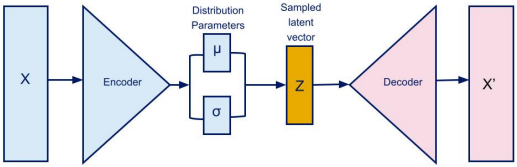
The magic of amortized inference!

- Kingma+Welling 2013, Rezende et al 2013:
- Decoder: (example continuous observations) $\mathbf{z} \to \mathbf{x}$

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\underbrace{\boldsymbol{\mu}_\theta(\mathbf{z})}_{\text{FFNN}}, \underbrace{\text{diag}(\boldsymbol{\sigma}^2_\theta(\mathbf{z}))}_{\text{FFNN}})$$

- Encoder: $\mathbf{x} \to \mathbf{z}$

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\underbrace{\boldsymbol{\mu}_\phi(\mathbf{x})}_{\text{FFNN}}, \underbrace{\text{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))}_{\text{FFNN}})$$



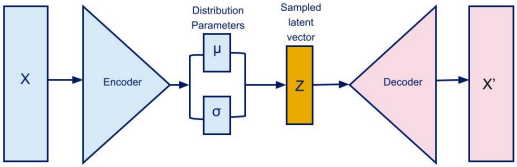
The magic of amortized inference!

- Kingma+Welling 2013, Rezende et al 2013:
- Decoder: (example continuous observations) $\mathbf{z} \to \mathbf{x}$

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\underbrace{\boldsymbol{\mu}_\theta(\mathbf{z})}_{\text{FFNN}}, \underbrace{\text{diag}(\boldsymbol{\sigma}^2_\theta(\mathbf{z}))}_{\text{FFNN}})$$

- Encoder: $\mathbf{x} \to \mathbf{z}$

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\underbrace{\boldsymbol{\mu}_\phi(\mathbf{x})}_{\text{FFNN}}, \underbrace{\text{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))}_{\text{FFNN}})$$



- Amortization = learn a mapping $q_\phi(\mathbf{z}|\mathbf{x})$ used for each data point rather than
- performing the computation $p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$ for each data point.

Log likelihood lower bound (aka ELBO)

- Bounding the log likelihood

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} && \leftarrow \text{exact} \\
&= \log \int \frac{q(\mathbf{z}|\mathbf{x})}{q(\mathbf{z}|\mathbf{x})} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} && \leftarrow \text{multiply by 1} \\
&= \log \int q(\mathbf{z}|\mathbf{x}) \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} d\mathbf{z} && \leftarrow \text{rearrange} \\
&\geq \int q(\mathbf{z}|\mathbf{x}) \log \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} d\mathbf{z} && \leftarrow \text{Jensen's inequality} \\
&= \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right] \equiv \mathcal{L}(\mathbf{x}) && \leftarrow \text{expectation notation}
\end{aligned}
$$

Log likelihood lower bound (aka ELBO)

- Bounding the log likelihood

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} && \leftarrow \text{exact} \\
&= \log \int \frac{q(\mathbf{z}|\mathbf{x})}{q(\mathbf{z}|\mathbf{x})} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} && \leftarrow \text{multiply by 1} \\
&= \log \int q(\mathbf{z}|\mathbf{x}) \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} d\mathbf{z} && \leftarrow \text{rearrange} \\
&\geq \int q(\mathbf{z}|\mathbf{x}) \log \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} d\mathbf{z} && \leftarrow \text{Jensen's inequality} \\
&= \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right] \equiv \mathcal{L}(\mathbf{x}) && \leftarrow \text{expectation notation}
\end{aligned}
$$

- Jensen's inequality - for any concave function ϕ, for example $\log$:

$$\phi\left(\mathbb{E}\left[\mathbf{x}\right]\right) \geq \mathbb{E}\left[\phi[\mathbf{x}]\right]$$

Deep variational autoencoders

- Variational objective - optimize $\sum_i \mathcal{L}_{\theta,\phi}(\mathbf{x}_i)$ wrt θ, ϕ:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(z)}{q_\phi(\mathbf{z}|\mathbf{x})}\right]$$

$$= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right]}_{\mathbb{E}_q[\text{log likelihood}]} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]}_{\text{regularization}}$$

Deep variational autoencoders

- Variational objective - optimize $\sum_i \mathcal{L}_{\theta,\phi}(\mathbf{x}_i)$ wrt θ, ϕ:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(z)}{q_\phi(\mathbf{z}|\mathbf{x})}\right]$$

$$= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right]}_{\mathbb{E}_q[\text{log likelihood}]} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]}_{\text{regularization}}$$

- Handle integration by sampling

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}^r)p(\mathbf{z}^r)}{q_\phi(\mathbf{z}^r|\mathbf{x})},$$

$$\mathbf{z}^r = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \otimes \boldsymbol{\epsilon}^r$$

Deep variational autoencoders

- Variational objective - optimize $\sum_i \mathcal{L}_{\theta,\phi}(\mathbf{x}_i)$ wrt θ, ϕ:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(z)}{q_\phi(\mathbf{z}|\mathbf{x})}\right]$$

$$= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right]}_{\mathbb{E}_q[\text{log likelihood}]} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]}_{\text{regularization}}$$

- Handle integration by sampling

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}^r)p(\mathbf{z}^r)}{q_\phi(\mathbf{z}^r|\mathbf{x})},$$

$$\mathbf{z}^r = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \otimes \boldsymbol{\epsilon}^r$$

- Auto-encoder: map "$\mathbf{x} \to \mathbf{z} \to \mathbf{x}$"

Deep variational autoencoders

- Variational objective - optimize $\sum_i \mathcal{L}_{\theta,\phi}(\mathbf{x}_i)$ wrt θ, ϕ:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(z)}{q_\phi(\mathbf{z}|\mathbf{x})}\right]$$
$$= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right]}_{\mathbb{E}_q[\text{log likelihood}]} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]}_{\text{regularization}}$$

- Handle integration by sampling

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}^r)p(\mathbf{z}^r)}{q_\phi(\mathbf{z}^r|\mathbf{x})},$$
$$\mathbf{z}^r = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \otimes \boldsymbol{\epsilon}^r$$

- Auto-encoder: map "$\mathbf{x} \to \mathbf{z} \to \mathbf{x}$"
- This is a variational auto-encoder (VAE):

$$\text{Encoder} \quad \mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \otimes \boldsymbol{\epsilon}$$
$$\text{Decoder} \quad p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mu_\theta(\mathbf{z}), \text{diag}(\sigma_\theta^2(\mathbf{z})))$$

Part 2:
Hierarchical VAE

Hierarchical VAE

- From 1 to $T+1$ latent layers:

$$\mathbf{z} \qquad \Rightarrow \qquad \mathbf{z}_0 \rightarrow \mathbf{z}_{\Delta t} \rightarrow \ldots \rightarrow \mathbf{z}_1$$

- Likelihood function

$$p(\mathbf{x}|\mathbf{z}) \qquad \Rightarrow \qquad p(\mathbf{x}|\mathbf{z}_1)$$

Hierarchical VAE

- From 1 to $T+1$ latent layers:

$$\mathbf{z} \qquad \Rightarrow \qquad \mathbf{z}_0 \to \mathbf{z}_{\Delta t} \to \ldots \to \mathbf{z}_1$$

- Likelihood function

$$p(\mathbf{x}|\mathbf{z}) \qquad \Rightarrow \qquad p(\mathbf{x}|\mathbf{z}_1)$$

- In terms of distributions:

$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$$

  is replaced by

$$p(\mathbf{x}, \mathbf{z}_0, \ldots, \mathbf{z}_1) = p(\mathbf{x}|\mathbf{z}_1) \prod_{l=1}^{T} p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})p(\mathbf{z}_0)$$

  with $t_l = l\Delta t$ and $\Delta t = 1/T$. (This choice will be more clear later!)

Hierarchical VAE

- From 1 to $T+1$ latent layers:

$$\mathbf{z} \qquad \Rightarrow \qquad \mathbf{z}_0 \to \mathbf{z}_{\Delta t} \to \ldots \to \mathbf{z}_1$$

- Likelihood function

$$p(\mathbf{x}|\mathbf{z}) \qquad \Rightarrow \qquad p(\mathbf{x}|\mathbf{z}_1)$$

- In terms of distributions:

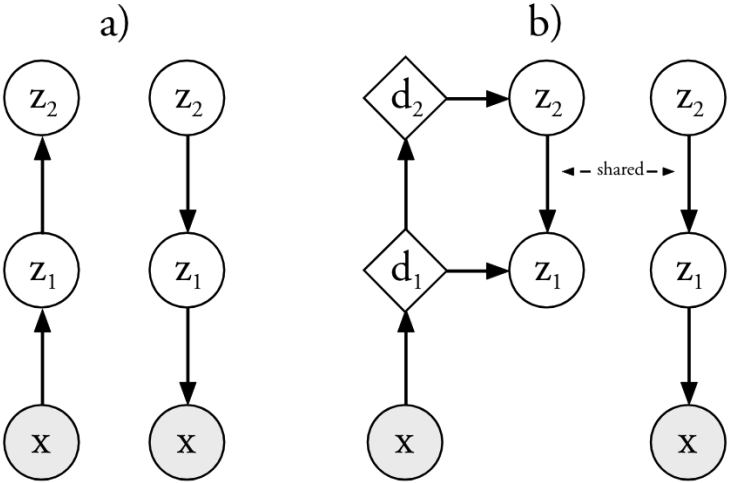
$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$$

  is replaced by

$$p(\mathbf{x}, \mathbf{z}_0, \ldots, \mathbf{z}_1) = p(\mathbf{x}|\mathbf{z}_1) \prod_{l=1}^{T} p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})p(\mathbf{z}_0)$$

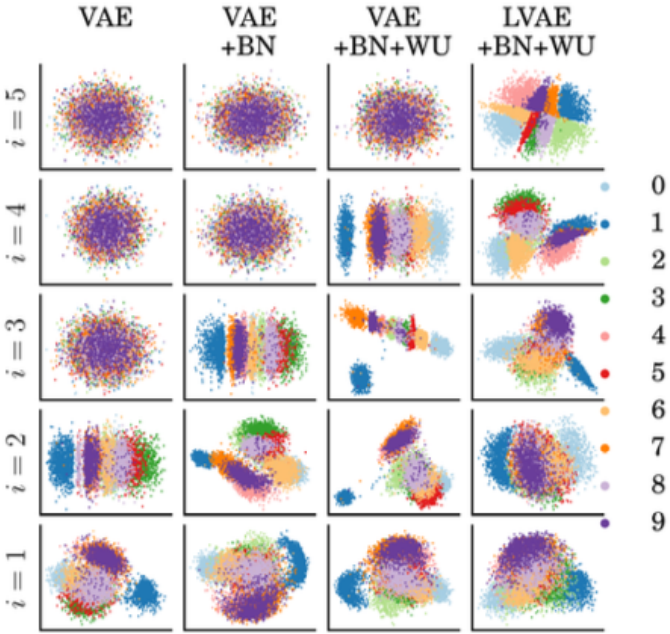
  with $t_l = l\Delta t$ and $\Delta t = 1/T$. (This choice will be more clear later!)
- But why do this?
  - It works better - more flexible distribution with deeper hierarchies (Assignment 4.1)
  - We can take the $T \to \infty$ and $\Delta t \to 0$ limit and get differential equations. Go from layers to smooth infinitesimal transformations. (Assigment 4.2 and 4.3)

The ladder VAE



Ladder VAE, C. K. Sønderby et al, 2016

The ladder VAE learned hierarchy for MNIST



Hierarchical VAE variational and ELBO

- Copy structure to the variational distribution:

$$p(\mathbf{z}_0, \ldots, \mathbf{z}_1) = \prod_{l=1}^{T} p(\mathbf{z}_{t_l} | \mathbf{z}_{t_{l-1}}) p(\mathbf{z}_0)$$

$$q(\mathbf{z}_0, \ldots, \mathbf{z}_1 | \mathbf{x}) = \prod_{l=1}^{T} q(\mathbf{z}_{t_l} | \mathbf{z}_{t_{l-1}}, \mathbf{x}) q(\mathbf{z}_0 | \mathbf{x})$$

- Plug into the ELBO:

$$\mathcal{L}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}_0, \ldots, \mathbf{z}_1 | \mathbf{x})} \left[\log \frac{p(\mathbf{x} | \mathbf{z}_1) p(\mathbf{z}_0, \ldots, \mathbf{z}_1)}{q(\mathbf{z}_0, \ldots, \mathbf{z}_1 | \mathbf{x})} \right]$$

Hierarchical VAE variational and ELBO

- Copy structure to the variational distribution:

$$p(\mathbf{z}_0, \ldots, \mathbf{z}_1) = \prod_{l=1}^{T} p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}}) p(\mathbf{z}_0)$$

$$q(\mathbf{z}_0, \ldots, \mathbf{z}_1|\mathbf{x}) = \prod_{l=1}^{T} q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}}, \mathbf{x}) q(\mathbf{z}_0|\mathbf{x})$$

- Plug into the ELBO:

$$\mathcal{L}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}_0,\ldots,\mathbf{z}_1|\mathbf{x})} \left[\log \frac{p(\mathbf{x}|\mathbf{z}_1) p(\mathbf{z}_0, \ldots, \mathbf{z}_1)}{q(\mathbf{z}_0, \ldots, \mathbf{z}_1|\mathbf{x})} \right]$$

- Use marginalization property $\mathbb{E}_{p(a,b)}[F(a)] = \mathbb{E}_{p(a)}[F(a)]$ - term names will be clear later

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})} [\log p(\mathbf{x}|\mathbf{z}_1)]}_{\text{reconstruction}} +$$

$$\underbrace{\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l}, \mathbf{z}_{t_{l-1}}|\mathbf{x})} \left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}}, \mathbf{x})} \right]}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})} \left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})} \right]}_{\text{prior}}$$

Hierarchical VAE algorithm

- Handle averaging by sampling:

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}_1^r)p(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r)}{q_\phi(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r|\mathbf{x})},$$

- Initialize: $\mathbf{z}_0^r \sim q(\mathbf{z}_0|\mathbf{x})$

$$\mathbf{z}_0^r = \boldsymbol{\mu}_{\phi,0}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi,0}(\mathbf{x}) \otimes \boldsymbol{\epsilon}_0^r$$

- For $l = 1,\ldots,T$: $\mathbf{z}_{t_l}^r \sim q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})$

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) \otimes \boldsymbol{\epsilon}_{t_l}^r$$

Hierarchical VAE algorithm

- Handle averaging by sampling:

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}_1^r)p(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r)}{q_\phi(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r|\mathbf{x})},$$

- Initialize: $\mathbf{z}_0^r \sim q(\mathbf{z}_0|\mathbf{x})$

$$\mathbf{z}_0^r = \boldsymbol{\mu}_{\phi,0}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi,0}(\mathbf{x}) \otimes \boldsymbol{\epsilon}_0^r$$

- For $l = 1,\ldots,T$: $\mathbf{z}_{t_l}^r \sim q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})$

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) \otimes \boldsymbol{\epsilon}_{t_l}^r$$

- Auto-encoder: map "$\mathbf{x} \to \mathbf{z} \to \mathbf{x}$"

Hierarchical VAE algorithm

- Handle averaging by sampling:

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}_1^r)p(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r)}{q_\phi(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r|\mathbf{x})},$$

- Initialize: $\mathbf{z}_0^r \sim q(\mathbf{z}_0|\mathbf{x})$

$$\mathbf{z}_0^r = \boldsymbol{\mu}_{\phi,0}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi,0}(\mathbf{x}) \otimes \boldsymbol{\epsilon}_0^r$$

- For $l = 1,\ldots,T$: $\mathbf{z}_{t_l}^r \sim q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})$

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) \otimes \boldsymbol{\epsilon}_{t_l}^r$$

- Auto-encoder: map "$\mathbf{x} \to \mathbf{z} \to \mathbf{x}$"
- This is a hierarchical variational auto-encoder (VAE):

$$\begin{aligned} &\text{Encoder} \quad \mathbf{z}_0,\ldots,\mathbf{z}_1 \sim q_\phi(\mathbf{z}_0,\ldots,\mathbf{z}_1|\mathbf{x}) \\ &\text{Decoder} \quad p_\theta(\mathbf{x}|\mathbf{z}_1) \end{aligned}$$

Hierarchical VAE algorithm

- Handle averaging by sampling:

$$\mathcal{L}_{\theta,\phi}(\mathbf{x}) \approx \frac{1}{R}\sum_{r=1}^{R} \log \frac{p_\theta(\mathbf{x}|\mathbf{z}_1^r)p(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r)}{q_\phi(\mathbf{z}_0^r,\ldots,\mathbf{z}_1^r|\mathbf{x})},$$

- Initialize: $\mathbf{z}_0^r \sim q(\mathbf{z}_0|\mathbf{x})$

$$\mathbf{z}_0^r = \boldsymbol{\mu}_{\phi,0}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi,0}(\mathbf{x}) \otimes \boldsymbol{\epsilon}_0^r$$

- For $l = 1,\ldots,T$: $\mathbf{z}_{t_l}^r \sim q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})$

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}^r) \otimes \boldsymbol{\epsilon}_{t_l}^r$$

- Auto-encoder: map "$\mathbf{x} \to \mathbf{z} \to \mathbf{x}$"
- This is a hierarchical variational auto-encoder (VAE):

$$\begin{aligned} &\text{Encoder} \quad \mathbf{z}_0,\ldots,\mathbf{z}_1 \sim q_\phi(\mathbf{z}_0,\ldots,\mathbf{z}_1|\mathbf{x}) \\ &\text{Decoder} \quad p_\theta(\mathbf{x}|\mathbf{z}_1) \end{aligned}$$

- Sampling: $\mathbf{z}_0 \sim p_\theta(\mathbf{x}_0)$, for $l = 1,\ldots,T$: $\mathbf{z}_{t_l} \sim p_\theta(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})$, $\mathbf{x} \sim p_\theta(\mathbf{x}|\mathbf{z}_1)$.

Part 3:

From hierarchical VAE to differential equations

A closer look at the diffusion term

- Look at a one term in the sum- use notation $t_l = t$ and $t_{l-1} = s$ shorthand

$$\mathbb{E}_{q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})}\right]$$

- Conditional Gaussians - θ: generative and ϕ variational parameters:

$$p(\mathbf{z}_t|\mathbf{z}_s) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\theta,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\theta,t}(\mathbf{x}, \mathbf{z}_s)))$$
$$q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x}) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\phi,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\phi,t}(\mathbf{x}, \mathbf{z}_s)))$$

A closer look at the diffusion term

- Look at a one term in the sum- use notation $t_l = t$ and $t_{l-1} = s$ shorthand

$$\mathbb{E}_{q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})}\right]$$

- Conditional Gaussians - θ: generative and ϕ variational parameters:

$$p(\mathbf{z}_t|\mathbf{z}_s) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\theta,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\theta,t}(\mathbf{x}, \mathbf{z}_s)))$$
$$q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x}) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\phi,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\phi,t}(\mathbf{x}, \mathbf{z}_s)))$$

- $q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})$ expectation - decompose averaging as $q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x}) = q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})q(\mathbf{z}_s|\mathbf{x})$

A closer look at the diffusion term

- Look at a one term in the sum- use notation $t_l = t$ and $t_{l-1} = s$ shorthand

$$\mathbb{E}_{q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})}\right]$$

- Conditional Gaussians - θ: generative and ϕ variational parameters:

$$p(\mathbf{z}_t|\mathbf{z}_s) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\theta,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\theta,t}(\mathbf{x}, \mathbf{z}_s)))$$
$$q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x}) = \mathcal{N}(\mathbf{z}_t|\boldsymbol{\mu}_{\phi,t}(\mathbf{x}, \mathbf{z}_s), \mathrm{diag}(\boldsymbol{\sigma}^2_{\phi,t}(\mathbf{x}, \mathbf{z}_s)))$$

- $q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})$ expectation - decompose averaging as $q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x}) = q(\mathbf{z}_t|\mathbf{z}_s, \mathbf{x})q(\mathbf{z}_s|\mathbf{x})$

$$\mathbb{E}_{q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})}\left[\log q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})\right] =$$
$$-\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log 2\pi\sigma^2_{\phi,t,i}(\mathbf{x}, \mathbf{z}_s)\right] - \frac{1}{2}\mathbb{E}_{q(\mathbf{z}_t, \mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{(\mathbf{z}_t - \mu_{\phi,t,i}(\mathbf{z}_s))^2}{\sigma^2_{\phi,t,i}(\mathbf{x}, \mathbf{z}_s)}\right]$$
$$-\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log 2\pi\sigma^2_{\phi,t,i}(\mathbf{x}, \mathbf{z}_s)\right] - \frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \underbrace{\frac{\sigma^2_{\phi,t,i}(\mathbf{x}, \mathbf{z}_s)}{\sigma^2_{\phi,t,i}(\mathbf{x}, \mathbf{z}_s)}}_{=1}\right]$$

A closer look at the diffusion term part 2

- $\log p(\mathbf{z}_t|\mathbf{z}_s)$ expectation - difficult term:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\theta,t,i}(\mathbf{z}_s))^2\right]$$

A closer look at the diffusion term part 2

- $\log p(\mathbf{z}_t|\mathbf{z}_s)$ expectation - difficult term:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\theta,t,i}(\mathbf{z}_s))^2\right]$$

- Final result:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log p(\mathbf{z}_t|\mathbf{z}_s)\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log 2\pi\sigma^2_{\theta,t,i}(\mathbf{x},\mathbf{z}_s)\right] - \frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) + (\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

A closer look at the diffusion term part 2

- $\log p(\mathbf{z}_t|\mathbf{z}_s)$ expectation - difficult term:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\theta,t,i}(\mathbf{z}_s))^2\right]$$

- Final result:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log p(\mathbf{z}_t|\mathbf{z}_s)\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log 2\pi\sigma^2_{\theta,t,i}(\mathbf{x},\mathbf{z}_s)\right]$$
$$-\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) + (\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

- Used add-subtract trick

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\theta,t,i}(\mathbf{z}_s))^2\right] = \mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) + \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) - \mu_{\theta,t,i}(\mathbf{z}_s))^2\right]$$

- Multiply out - cross term disappears:

$$2\,\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[(z_{t,i} - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))(\mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) - \mu_{\theta,t,i}(\mathbf{z}_s))\right] =$$
$$2\,\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\underbrace{\mathbb{E}_{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\left[(z_{t,i} - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))\right]}_{=0}(\mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) - \mu_{\theta,t,i}(\mathbf{z}_s))\right] = 0$$

Collecting all terms

- Diffusion term from ELBO

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right]$$

- Putting it all together:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log \frac{\sigma^2_{\theta,t,i}(\mathbf{x},\mathbf{z}_s)}{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s)}\right]$$
$$-\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) - \sigma^2_{\theta,t,i}(\mathbf{z}_s) + (\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

Collecting all terms

- Diffusion term from ELBO

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right]$$

- Putting it all together:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \log \frac{\sigma^2_{\theta,t,i}(\mathbf{x},\mathbf{z}_s)}{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s)}\right]$$
$$-\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{\sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s) - \sigma^2_{\theta,t,i}(\mathbf{z}_s) + (\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

- Let us do a major simplification $\sigma_{\theta,t,i}(\mathbf{z}_s) = \sigma^2_{\phi,t,i}(\mathbf{x},\mathbf{z}_s)$:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{(\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

- The diffusion term is now a just a sum over weighted squared mean differences.

Prepare for ∞ layers

- One layer in the diffusion term:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{(\mu_{\theta,t,i}(\mathbf{z}_s) - \mu_{\phi,t,i}(\mathbf{x},\mathbf{z}_s))^2}{\sigma^2_{\theta,t,i}(\mathbf{z}_s)}\right]$$

- Prepare to go to ∞ layers using a new paramterization - remember $\Delta t = 1/T$:

$$\boldsymbol{\mu}_{\theta,t,}(\mathbf{z}) = \mathbf{z} + \mathbf{f}(\mathbf{z},t)\Delta t$$
$$\boldsymbol{\mu}_{\phi,t}(\mathbf{x},\mathbf{z}) = \mathbf{z} + \mathbf{g}(\mathbf{x},\mathbf{z},t)\Delta t$$
$$\boldsymbol{\sigma}_{\theta,t}(\mathbf{z}) = \boldsymbol{\sigma}(\mathbf{z},t)\sqrt{\Delta t}$$

- Notice Δt:

$$\mathbb{E}_{q(\mathbf{z}_t,\mathbf{z}_s|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_t|\mathbf{z}_s)}{q(\mathbf{z}_t|\mathbf{z}_s,\mathbf{x})}\right] = -\frac{1}{2}\mathbb{E}_{q(\mathbf{z}_s|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_s,s) - g_i(\mathbf{x},\mathbf{z}_s,s))^2}{\sigma_i^2(\mathbf{z}_s,t)} \underbrace{\frac{(\Delta t)^2}{\Delta t}}_{=\Delta t}\right]$$

∞ layers limit

- Hierarchical VAE ELBO again:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}} + \underbrace{\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l},\mathbf{z}_{t_{l-1}}|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})}\right]}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

∞ layers limit

- Hierarchical VAE ELBO again:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}} + \underbrace{\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l},\mathbf{z}_{t_{l-1}}|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})}\right]}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- Use our result for the diffusion term:

$$\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l},\mathbf{z}_{t_{l-1}}|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})}\right] = -\frac{1}{2}\sum_{l=0}^{T-1} \mathbb{E}_{q(\mathbf{z}_{t_l}|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_{t_l},t_l) - g_i(\mathbf{x},\mathbf{z}_{t_l},t_l))^2}{\sigma_i^2(\mathbf{z}_{t_l},t_l)}\right]\Delta t$$

∞ layers limit

- Hierarchical VAE ELBO again:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}} + \underbrace{\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l},\mathbf{z}_{t_{l-1}}|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})}\right]}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- Use our result for the diffusion term:

$$\sum_{l=1}^{T} \mathbb{E}_{q(\mathbf{z}_{t_l},\mathbf{z}_{t_{l-1}}|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}})}{q(\mathbf{z}_{t_l}|\mathbf{z}_{t_{l-1}},\mathbf{x})}\right] = -\frac{1}{2}\sum_{l=0}^{T-1} \mathbb{E}_{q(\mathbf{z}_{t_l}|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_{t_l},t_l) - g_i(\mathbf{x},\mathbf{z}_{t_l},t_l))^2}{\sigma_i^2(\mathbf{z}_{t_l},t_l)}\right]\Delta t$$

- Recall $t_l = l/T$. Take $\Delta t \to 0$ limit - from sum to integral $\sum_{l=0}^{T-1} F(t_l)\Delta t \to \int_0^1 F(t)dt$:

$$\to -\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt$$

Part 4:

Diffusion, score matching, neural ordinary differential equations and flow matching

Roadmap for the remainder of the lecture

- Diffusion models - stochastic differential equations - DDPM - finite step version and VDM -∞ step version
- Score matching and earlier work Denoising score matching
- Ordinary differential equations - Neural ODE
- Flow matching
- Final remarks - Why bother spending time on deep learning

Part 4.1:
Diffusion models

ELBO and stochastic differential equations

- From the ∞-layer VAE we get the diffusion model ELBO

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

ELBO and stochastic differential equations

- From the ∞-layer VAE we get the diffusion model ELBO

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- Let us revisit the encoder in the variational distribution:

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) \otimes \boldsymbol{\epsilon}_{t_l}$$

ELBO and stochastic differential equations

- From the ∞-layer VAE we get the diffusion model ELBO

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- Let us revisit the encoder in the variational distribution:

$$\mathbf{z}_{t_l}^r = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) \otimes \boldsymbol{\epsilon}_{t_l}$$

- This can now be written as a so-called *stochastic differential equation (SDE)*

$$\mathbf{z}_{t+dt} = \underbrace{\mathbf{z}_t + \mathbf{g}(\mathbf{x},\mathbf{z}_t,t)dt}_{\boldsymbol{\mu}_{\phi,t}(\mathbf{x},\mathbf{z}_t)} + \underbrace{\sqrt{dt}\,\boldsymbol{\sigma}(\mathbf{z}_t,t)}_{\boldsymbol{\sigma}_{\phi,t}(\mathbf{x},\mathbf{z}_t)} \otimes \boldsymbol{\epsilon}_t$$

ELBO and stochastic differential equations

- From the ∞-layer VAE we get the diffusion model ELBO

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- Let us revisit the encoder in the variational distribution:

$$\mathbf{z}^r_{t_l} = \boldsymbol{\mu}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) + \boldsymbol{\sigma}_{\phi,t_l}(\mathbf{x},\mathbf{z}_{t_{l-1}}) \otimes \boldsymbol{\epsilon}_{t_l}$$

- This can now be written as a so-called *stochastic differential equation (SDE)*

$$\mathbf{z}_{t+dt} = \underbrace{\mathbf{z}_t + \mathbf{g}(\mathbf{x},\mathbf{z}_t,t)dt}_{\boldsymbol{\mu}_{\phi,t}(\mathbf{x},\mathbf{z}_t)} + \underbrace{\sqrt{dt}\,\boldsymbol{\sigma}(\mathbf{z}_t,t)}_{\boldsymbol{\sigma}_{\phi,t}(\mathbf{x},\mathbf{z}_t)} \otimes \boldsymbol{\epsilon}_t$$

- We can also write the generative model like that:

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \mathbf{f}(\mathbf{z}_t,t)dt + \sqrt{dt}\,\boldsymbol{\sigma}(\mathbf{z}_t,t) \otimes \boldsymbol{\epsilon}_t$$

- In SDEs $d\boldsymbol{W} := \sqrt{dt}\boldsymbol{\epsilon} \sim \mathcal{N}(d\boldsymbol{W}|\mathbf{0}, dt\boldsymbol{I})$ is called the Wiener process.

Diffusion model - let us turn this into an algorithm

- Final result for the ELBO:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- How to turn this into an efficient algorithm:
  - Define likelihood $p(\mathbf{x}|\mathbf{z}_1)$ and prior $p(\mathbf{z}_0)$ - easy!
  - Define $\mathbf{f}$ and $\boldsymbol{\sigma}$ as neural networks - easy!

Diffusion model - let us turn this into an algorithm

- Final result for the ELBO:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- How to turn this into an efficient algorithm:
  - Define likelihood $p(\mathbf{x}|\mathbf{z}_1)$ and prior $p(\mathbf{z}_0)$ - easy!
  - Define $\mathbf{f}$ and $\boldsymbol{\sigma}$ as neural networks - easy!
  - Choose $q(\mathbf{z}_0|\mathbf{x})$ to be Gaussian and $\mathbf{g}$ linear $\mathbf{z}$, then $q(\mathbf{z}_t|\mathbf{x})$ is Gaussian with known mean and variance (see Bonus material)

Diffusion model - let us turn this into an algorithm

- Final result for the ELBO:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$
$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- How to turn this into an efficient algorithm:
  - Define likelihood $p(\mathbf{x}|\mathbf{z}_1)$ and prior $p(\mathbf{z}_0)$ - easy!
  - Define $\mathbf{f}$ and $\boldsymbol{\sigma}$ as neural networks - easy!
  - Choose $q(\mathbf{z}_0|\mathbf{x})$ to be Gaussian and $\mathbf{g}$ linear $\mathbf{z}$, then $q(\mathbf{z}_t|\mathbf{x})$ is Gaussian with known mean and variance (see Bonus material)
  - This avoids needing to simulate the SDE: $\mathbf{z}_0 \sim q(z_0|\mathbf{x})$ and for $t = 0, dt, \ldots, 1 - dt$:

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \mathbf{f}(\mathbf{z}_t, t)dt + \boldsymbol{\sigma}(\mathbf{z}_t, t) \otimes d\boldsymbol{W}_t$$

Diffusion model - let us turn this into an algorithm

- Final result for the ELBO:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log p(\mathbf{x}|\mathbf{z}_1)\right]}_{\text{reconstruction}}$$

$$\underbrace{-\frac{1}{2}\int_0^1 \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\left[\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{x},\mathbf{z}_t,t))^2}{\sigma_i^2(\mathbf{z}_t,t)}\right] dt}_{\text{diffusion}} + \underbrace{\mathbb{E}_{q(\mathbf{z}_0|\mathbf{x})}\left[\log \frac{p(\mathbf{z}_0)}{q(\mathbf{z}_0|\mathbf{x})}\right]}_{\text{prior}}$$

- How to turn this into an efficient algorithm:
  - Define likelihood $p(\mathbf{x}|\mathbf{z}_1)$ and prior $p(\mathbf{z}_0)$ - easy!
  - Define $\mathbf{f}$ and $\boldsymbol{\sigma}$ as neural networks - easy!
  - Choose $q(\mathbf{z}_0|\mathbf{x})$ to be Gaussian and $\mathbf{g}$ linear $\mathbf{z}$, then $q(\mathbf{z}_t|\mathbf{x})$ is Gaussian with known mean and variance (see Bonus material)
  - This avoids needing to simulate the SDE: $\mathbf{z}_0 \sim q(z_0|\mathbf{x})$ and for $t = 0, dt, \ldots, 1 - dt$:

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \mathbf{f}(\mathbf{z}_t, t)dt + \boldsymbol{\sigma}(\mathbf{z}_t, t) \otimes d\boldsymbol{W}_t$$

  - Simulation-free estimation of diffusion term - replace integral over t with sample

$$\int_0^1 M(t)dt = \mathbb{E}_{\mathcal{U}([0,1])}[M(t)] \to M(t') \quad \text{with} \quad t' \sim \mathcal{U}[0,1]$$

SDE Variational pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), $\mathbf{g}$ (variational drift function), $\boldsymbol{\sigma}$ (diffusion), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

SDE Variational pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), $\mathbf{g}$ (variational drift function), $\boldsymbol{\sigma}$ (diffusion), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

Def `SDESample`($\mathbf{z}_{\text{init}}$, t_{init}, Δt)**:**

  $\mathbf{z}, t = \mathbf{z}_{\text{init}}, t_{\text{init}}$

  for *i=1,...,T* **do**

    $\Delta \boldsymbol{W} \sim \mathcal{N}(\mathbf{0}, \Delta t \boldsymbol{I})$

    $\mathbf{z} = \mathbf{z} + \mathbf{f}(\mathbf{z}, t)\Delta t + \boldsymbol{\sigma}(\mathbf{z}, t) \otimes \Delta \boldsymbol{W}$

    $t = t + \Delta t$

  return $\mathbf{z}$

SDE Variational pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), $\mathbf{g}$ (variational drift function), $\boldsymbol{\sigma}$ (diffusion), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

Def `SDESample(`$\mathbf{z}_{\text{init}}$, t_{init}, Δt`):`

  $\mathbf{z}, t = \mathbf{z}_{\text{init}}, t_{\text{init}}$

  for *i=1,...,T* **do**

    $\Delta \boldsymbol{W} \sim \mathcal{N}(\mathbf{0}, \Delta t \boldsymbol{I})$

    $\mathbf{z} = \mathbf{z} + \mathbf{f}(\mathbf{z}, t)\Delta t + \boldsymbol{\sigma}(\mathbf{z}, t) \otimes \Delta \boldsymbol{W}$

    $t = t + \Delta t$

  return $\mathbf{z}$

Def `SDEELBO(`$\mathbf{x}$`):`

  $t \sim \mathcal{U}[0, 1]$

  $\mathbf{z}_0, \mathbf{z}_t, \mathbf{z}_1 \sim q_0(\mathbf{z}_0|y), q_t(\mathbf{z}_t|y), q_1(\mathbf{z}_1|y)$

  $\mathcal{L} = \log p(\mathbf{x}|\mathbf{z}_1) + \log p_0(\mathbf{z}_0) - \log q_0(\mathbf{z}_0|y) - \frac{1}{2}\sum_i \frac{(f_i(\mathbf{z}_t,t) - g_i(\mathbf{z}_t,\mathbf{x},t))^2}{\sigma_i^2(\mathbf{z}_t,t)}$

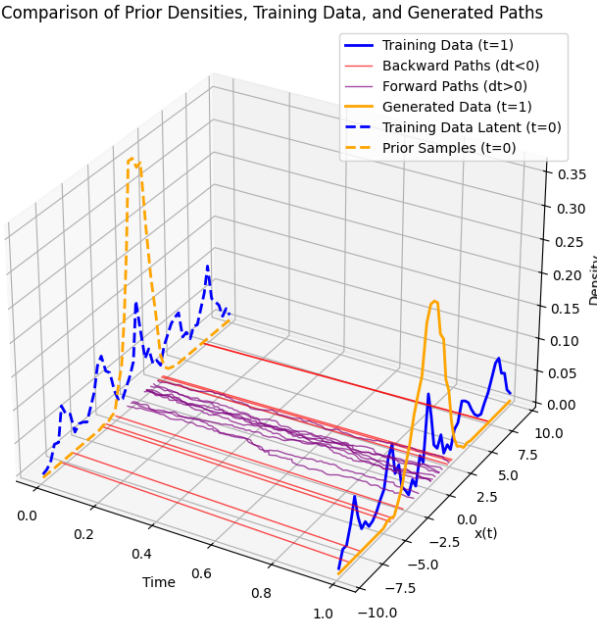
  return $\mathcal{L}$

SDE Variational pseudo code

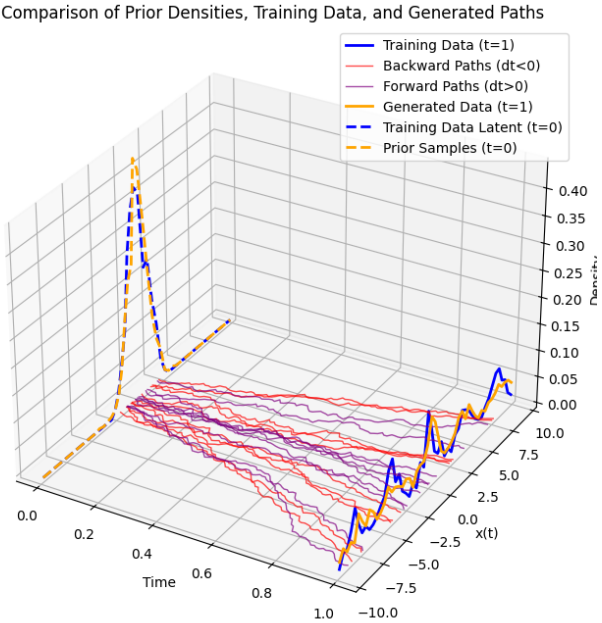
Input: T (number of discretization steps), $\mathbf{f}$ (drift function), $\mathbf{g}$ (variational drift function), $\boldsymbol{\sigma}$ (diffusion), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

```
Def SDESample(z_init, t_init, Δt):
    z, t = z_init, t_init
    for i=1,...,T do
        ΔW ~ N(0, ΔtI)
        z = z + f(z, t)Δt + σ(z, t) ⊗ ΔW
        t = t + Δt
    return z
Def SDEELBO(x):
    t ~ U[0, 1]
    z_0, z_t, z_1 ~ q_0(z_0|y), q_t(z_t|y), q_1(z_1|y)
    L = log p(x|z_1) + log p_0(z_0) − log q_0(z_0|y) − 1/2 Σ_i (f_i(z_t,t) − g_i(z_t,x,t))^2 / σ_i^2(z_t,t)
    return L
Def Sample():
    z_0 ~ p_0(z_0)
    z_1 = SDESample(z_init = z_0, t_init = 0, Δt = 1/T)
    x_sample ~ p(x_sample|z_1)
    return x_sample
```

SDE 1d example at initialization



SDE 1d example after training



Part 4.2:
Score matching

Score matching

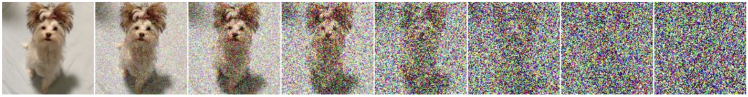
- Confession: There is much more to stochastic differential equations. . . .

Score matching

- Confession: There is much more to stochastic differential equations. . . .
- Score matching - we can work with both forward and reverse-time processes

Forward SDE (data → noise)

$$\mathbf{x}(0) \longrightarrow \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{w} \longrightarrow \mathbf{x}(T)$$



score function

$$\mathbf{x}(0) \longleftarrow \mathrm{d}\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g^2(t)\boxed{\nabla_{\mathbf{x}} \log p_t(\mathbf{x})}\right]\mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}} \longleftarrow \mathbf{x}(T)$$

Reverse SDE (noise → data)

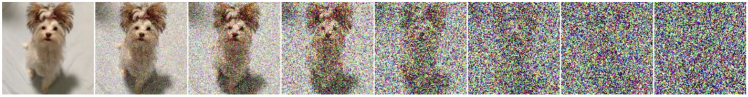
- The two processes have the *same path distributions*.
- ⇒ If we know the *score function*, then we can turn noise into data!

Score matching

- Confession: There is much more to stochastic differential equations. . . .
- Score matching - we can work with both forward and reverse-time processes

Forward SDE (data → noise)

$$\mathbf{x}(0) \longrightarrow \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{w} \longrightarrow \mathbf{x}(T)$$



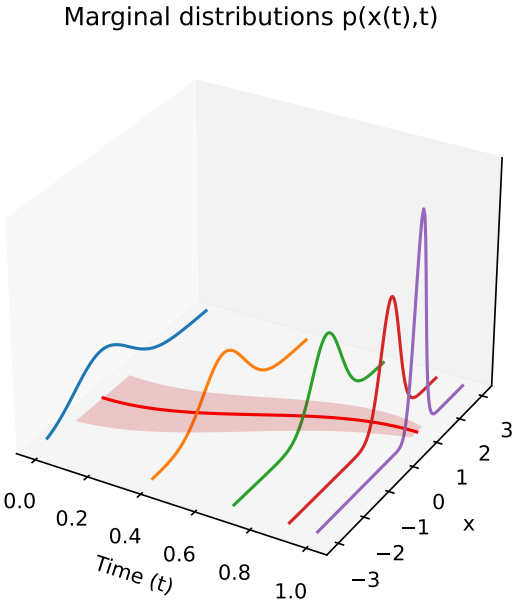
score function

$$\mathbf{x}(0) \longleftarrow \mathrm{d}\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g^2(t)\boxed{\nabla_{\mathbf{x}} \log p_t(\mathbf{x})}\right] \mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}} \longleftarrow \mathbf{x}(T)$$

Reverse SDE (noise → data)

- The two processes have the *same path distributions*.
- ⇒ If we know the *score function*, then we can turn noise into data!
- Follows from the Fokker-Planck equation: $\frac{\partial p_t(\mathbf{x})}{\partial t} = \ldots$ (see Bonus material)
- Next slides: Write this in our ∞-layer VAE notation.

The marginal



Legends change notation $x \to z$

Forward and reverse-time SDE for variational

- Reminder - forward $dt > 0$ variational process:
- Initialized at $t = 0$

$$\mathbf{z}_0 \sim q_0(\mathbf{z}_0|\mathbf{x})$$

Forward and reverse-time SDE for variational

- Reminder - forward $dt > 0$ variational process:
- Initialized at $t = 0$

$$\mathbf{z}_0 \sim q_0(\mathbf{z}_0|\mathbf{x})$$

- Run forward in time $dt > 0$ (with simplification $\boldsymbol{\sigma}(\mathbf{z}, t) \to \sigma(t)$)

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \mathbf{g}(\mathbf{x}, \mathbf{z}_t, t)dt + \sigma(t)d\boldsymbol{W}_t$$

- Wiener process - independent Gaussian noise $d\boldsymbol{W} \sim \mathcal{N}(d\boldsymbol{W}|\mathbf{0}, dt\boldsymbol{I})$.

Forward and reverse-time SDE for variational

- Reminder - forward $dt > 0$ variational process:
- Initialized at $t = 0$

$$\mathbf{z}_0 \sim q_0(\mathbf{z}_0|\mathbf{x})$$

- Run forward in time $dt > 0$ (with simplification $\boldsymbol{\sigma}(\mathbf{z}, t) \to \sigma(t)$)

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \mathbf{g}(\mathbf{x}, \mathbf{z}_t, t)dt + \sigma(t)d\boldsymbol{W}_t$$

- Wiener process - independent Gaussian noise $d\boldsymbol{W} \sim \mathcal{N}(d\boldsymbol{W}|\mathbf{0}, dt\boldsymbol{I})$.
- We can also define a so-called diffusion process that runs in reverse-time $dt < 0$:
- Initialized at $t = 1$ close to data:

$$\mathbf{z}_1 \sim q_1(\mathbf{z}_1|\mathbf{x})$$

- Run backwards in time $dt < 0$:

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \overleftarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t)dt + \sigma(t)d\overleftarrow{\boldsymbol{W}}_t$$

- Reverse-time Wiener process - independent Gaussian noise $d\overleftarrow{\boldsymbol{W}} \sim \mathcal{N}(d\overleftarrow{\boldsymbol{W}}|\mathbf{0}, |dt|\boldsymbol{I})$.

Relating forward and reverse-time drift

- The Fokker-Planck equation (Bonus material):
- A particular choice of $\mathbf{g}$ and $\overleftarrow{\mathbf{g}}$ makes the path distributions identical.

Relating forward and reverse-time drift

- The Fokker-Planck equation (Bonus material):
- A particular choice of $\mathbf{g}$ and $\overleftarrow{\mathbf{g}}$ makes the path distributions identical.
- Score matching is defined in terms of the diffusion with $\mathbf{x}$-independent drift:

$$\overleftarrow{\mathbf{g}}(\mathbf{z}_t, t)$$

- The equivalent forward drift is then given by:

$$\mathbf{g}(\mathbf{x}, \mathbf{z}_t, t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t, t) + \sigma^2(t) \nabla \log q_t(\mathbf{z}_t | x)$$

- $\nabla = \begin{pmatrix} \frac{\partial}{\partial z_1} \\ \frac{\partial}{\partial z_2} \\ \vdots \end{pmatrix}$ is the gradient vector and $\frac{\partial}{\partial z_i}$ is the partial derivative wrt to the ith element.

Diffusion ELBO term as score objective

- Reminder - minus the ELBO diffusion term with simplified $\sigma(t)$:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\mathbf{f}(\mathbf{z}_t,t)-\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- We can write the variational drift in terms of the reverse-time diffusion drift:

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|x)$$

- We can choose the generative drift in terms of a reverse-time drift:

$$\mathbf{f}(\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log p_t(\mathbf{z}_t)$$

Diffusion ELBO term as score objective

- Reminder - minus the ELBO diffusion term with simplified $\sigma(t)$:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\mathbf{f}(\mathbf{z}_t,t)-\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- We can write the variational drift in terms of the reverse-time diffusion drift:

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|x)$$

- We can choose the generative drift in terms of a reverse-time drift:

$$\mathbf{f}(\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log p_t(\mathbf{z}_t)$$

- The score function $\nabla \log p_t(\mathbf{z}_t)$ is unknown. Remember $p_t(\mathbf{z}_t)$ is what we want to learn!

Diffusion ELBO term as score objective

- Reminder - minus the ELBO diffusion term with simplified $\sigma(t)$:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\mathbf{f}(\mathbf{z}_t,t)-\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right] dt$$

- We can write the variational drift in terms of the reverse-time diffusion drift:

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|x)$$

- We can choose the generative drift in terms of a reverse-time drift:

$$\mathbf{f}(\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\nabla \log p_t(\mathbf{z}_t)$$

- The score function $\nabla \log p_t(\mathbf{z}_t)$ is unknown. Remember $p_t(\mathbf{z}_t)$ is what we want to learn!
- Replace it by a learnable function $\boldsymbol{s}_\theta(\mathbf{z},t)$: $\nabla \log p_t(\mathbf{z}_t) \to \boldsymbol{s}_\theta(\mathbf{z},t)$
- The score matching objective is minus the ELBO diffusion term:

$$\frac{1}{2}\int_0^1 \sigma^2(t)\,\mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[||\boldsymbol{s}_\theta(\mathbf{z},t) - \nabla \log q_t(\mathbf{z}_t|x)||^2\right] dt$$

- Generative model as before but now with drift function in terms of the score function:

$$\mathbf{f}(\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t) + \sigma^2(t)\boldsymbol{s}_\theta(\mathbf{z}_t,t)$$

Part 4.3:
Neural ordinary differential equations

Ordinary differential equation (ODE) - setting the stage

- Ordinary differential equations is the SDE's *deterministic* cousin
- ODE definition

$$d\mathbf{z} = \mathbf{f}(\mathbf{z}, t)dt$$

or traditional (ordinary) differential equation:

$$\frac{d\mathbf{z}}{dt} = \mathbf{f}(\mathbf{x}, t) = \mathbf{drift}$$

Ordinary differential equation (ODE) - setting the stage

- Ordinary differential equations is the SDE's *deterministic* cousin
- ODE definition

$$d\mathbf{z} = \mathbf{f}(\mathbf{z}, t)dt$$

  or traditional (ordinary) differential equation:

$$\frac{d\mathbf{z}}{dt} = \mathbf{f}(\mathbf{x}, t) = \mathbf{drift}$$

- Let us pick the equation apart
  - dt is an infinitesimal time step.
  - It can both be positive (forward) and negative (reverse-time)
  - When run on computer, it is finite and small, e.g. $dt = 1/100$
  - $d\mathbf{z}$ is the change in $\mathbf{z}$.
  - So if $\mathbf{z} = \mathbf{z}_t$ at t then:

$$d\mathbf{z}_t = \mathbf{z}_{t+dt} - \mathbf{z}_t = \mathbf{f}(\mathbf{x}_t, t)dt$$

ODE for generative modeling

- Sample prior, integrate and sample likelihood function:

$$
\begin{aligned}
\mathbf{z}_0 &\sim p_0(\mathbf{z}_0) \\
\mathbf{z}_{t+dt} &= \mathbf{z}_t + f(\mathbf{z}_t, t)dt, \quad \text{for} \quad t = 0, dt, 2dt, \ldots, 1 - dt \\
\mathbf{x} &\sim p(\mathbf{x}|\mathbf{z}_1)
\end{aligned}
$$

- Let us pick the steps apart:

ODE for generative modeling

- Sample prior, integrate and sample likelihood function:

$$
\begin{aligned}
\mathbf{z}_0 &\sim p_0(\mathbf{z}_0) \\
\mathbf{z}_{t+dt} &= \mathbf{z}_t + f(\mathbf{z}_t, t)dt, \quad \text{for} \quad t = 0, dt, 2dt, \ldots, 1 - dt \\
\mathbf{x} &\sim p(\mathbf{x}|\mathbf{z}_1)
\end{aligned}
$$

- Let us pick the steps apart:
  - $\sim$ means drawing a sample from the distribution.
  - So we start from drawing from the latent prior $p_0(\mathbf{z}_0)$.

ODE for generative modeling

- Sample prior, integrate and sample likelihood function:

$$\begin{aligned} \mathbf{z}_0 &\sim p_0(\mathbf{z}_0) \\ \mathbf{z}_{t+dt} &= \mathbf{z}_t + f(\mathbf{z}_t, t)dt, \quad \text{for} \quad t = 0, dt, 2dt, \ldots, 1 - dt \\ \mathbf{x} &\sim p(\mathbf{x}|\mathbf{z}_1) \end{aligned}$$

- Let us pick the steps apart:
  - $\sim$ means drawing a sample from the distribution.
  - So we start from drawing from the latent prior $p_0(\mathbf{z}_0)$.
  - The (Euler) integration steps corresponds to repeated application of a residual NN :

$$\mathbf{z}_{t'} = \mathbf{z}_t + \mathbf{F}(\mathbf{z}_t, t)\,, \quad t' = t + dt$$

  - $\mathbf{F}(\mathbf{z}, t) = \mathbf{f}(\mathbf{z}, t)dt$ uses parameter sharing but is layer-dependent due to t input.

ODE for generative modeling

- Sample prior, integrate and sample likelihood function:

$$\begin{aligned} \mathbf{z}_0 &\sim p_0(\mathbf{z}_0) \\ \mathbf{z}_{t+dt} &= \mathbf{z}_t + f(\mathbf{z}_t, t)dt, \quad \text{for} \quad t = 0, dt, 2dt, \ldots, 1 - dt \\ \mathbf{x} &\sim p(\mathbf{x}|\mathbf{z}_1) \end{aligned}$$

- Let us pick the steps apart:
  - $\sim$ means drawing a sample from the distribution.
  - So we start from drawing from the latent prior $p_0(\mathbf{z}_0)$.
  - The (Euler) integration steps corresponds to repeated application of a residual NN :

$$\mathbf{z}_{t'} = \mathbf{z}_t + \mathbf{F}(\mathbf{z}_t, t)\,, \quad t' = t + dt$$

  - $\mathbf{F}(\mathbf{z}, t) = \mathbf{f}(\mathbf{z}, t)dt$ uses parameter sharing but is layer-dependent due to t input.
  - There are also more advanced and costly integration - so-called Runge-Kutta (see Assignment 4.2).

Parameter learning objective - likelihood

- Training data: $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}$
- We want to use maximum likelihood so we need probability of data:

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}_1)p(\mathbf{Z})d\mathbf{Z}$$

- $\mathbf{Z} = \mathbf{z}_0, \mathbf{z}_{dt}, \mathbf{z}_{2dt}, \ldots, \mathbf{z}_1$ is the latent variables for all the layers stacked.

Parameter learning objective - likelihood

- Training data: $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}$
- We want to use maximum likelihood so we need probability of data:

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}_1)p(\mathbf{Z})d\mathbf{Z}$$

- $\mathbf{Z} = \mathbf{z}_0, \mathbf{z}_{dt}, \mathbf{z}_{2dt}, \ldots, \mathbf{z}_1$ is the latent variables for all the layers stacked.
- (This integral is not well-defined for $dt \to 0$ but everything works out fine in the end.)

Parameter learning objective - likelihood

- Training data: $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}$
- We want to use maximum likelihood so we need probability of data:

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}_1)p(\mathbf{Z})d\mathbf{Z}$$

- $\mathbf{Z} = \mathbf{z}_0, \mathbf{z}_{dt}, \mathbf{z}_{2dt}, \ldots, \mathbf{z}_1$ is the latent variables for all the layers stacked.
- (This integral is not well-defined for $dt \to 0$ but everything works out fine in the end.)
- Marginalize out $\mathbf{Z}$ apart from $\mathbf{z}_1$ (using Bayes' rule)

$$\begin{aligned} p(\mathbf{x}) &= \int p(\mathbf{x}|\mathbf{z}_1) \underbrace{p(\mathbf{z}_0, \ldots, \mathbf{z}_{1-dt}|\mathbf{z}_1)p_1(\mathbf{z}_1)}_{p(\mathbf{Z})} d\mathbf{Z} \\ &= \int p(\mathbf{x}|\mathbf{z}_1) \underbrace{\int p(\mathbf{z}_0, \ldots, \mathbf{z}_{1-dt}|\mathbf{z}_1)d\mathbf{z}_0, , \ldots, d\mathbf{z}_{1-dt}}_{=1} p_1(\mathbf{z}_1)d\mathbf{z}_1 = \int p(\mathbf{x}|\mathbf{z}_1)p_1(\mathbf{z}_1)d\mathbf{z}_1 \end{aligned}$$

- All we need is is the marginal at $t = 1$: $p_1(\mathbf{z}_1)$.

ELBO ODE

- Exact log likelihood for data point $\mathbf{x}$:

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x}|\mathbf{z}_1) p_1(\mathbf{z}_1) d\mathbf{z}_1$$

- We can lower bound it with variational distribution $q_1(\mathbf{z}_1|\mathbf{x})$:

$$\log p(\mathbf{x}) \geq \mathcal{L}_{\mathrm{ODE}}(\mathbf{x}) := \mathbb{E}_{q_1(\mathbf{z}_1|\mathbf{x})} \left[\log \frac{p(\mathbf{x}|\mathbf{z}_1) p_1(\mathbf{z}_1)}{q_1(\mathbf{z}_1|\mathbf{x})} \right]$$

ELBO ODE

- Exact log likelihood for data point $\mathbf{x}$:

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x}|\mathbf{z}_1) p_1(\mathbf{z}_1) d\mathbf{z}_1$$

- We can lower bound it with variational distribution $q_1(\mathbf{z}_1|\mathbf{x})$:

$$\log p(\mathbf{x}) \geq \mathcal{L}_{\mathrm{ODE}}(\mathbf{x}) := \mathbb{E}_{q_1(\mathbf{z}_1|\mathbf{x})} \left[\log \frac{p(\mathbf{x}|\mathbf{z}_1) p_1(\mathbf{z}_1)}{q_1(\mathbf{z}_1|\mathbf{x})} \right]$$

- Training objective $\sum_{i=1}^{n} \mathcal{L}_{\mathrm{ODE}}(\mathbf{x}^{(i)})$

Ordinary differential equation for $\log p_t(x_t)$ - Liouville

- We still need to be able to compute $p_1(\mathbf{z}_1)$
- Use Liouville equation (Bonus material)

$$\frac{\partial p_t(z_t)}{\partial t} = -\nabla \cdot [\mathbf{f}(\mathbf{z}_t, t) p_t(\mathbf{z}_t)]$$

- Divergence of vector (field) gives a scalar output: $\nabla \cdot \mathbf{f} = \sum_j \frac{\partial f_j(\mathbf{z},t)}{\partial z_j}$

Ordinary differential equation for $\log p_t(x_t)$ - Liouville

- We still need to be able to compute $p_1(\mathbf{z}_1)$
- Use Liouville equation (Bonus material)

$$\frac{\partial p_t(z_t)}{\partial t} = -\nabla \cdot [\mathbf{f}(\mathbf{z}_t, t) p_t(\mathbf{z}_t)]$$

- Divergence of vector (field) gives a scalar output: $\nabla \cdot \mathbf{f} = \sum_j \frac{\partial f_j(\mathbf{z},t)}{\partial z_j}$
- Total derivative versus partial derivative :

$$\underbrace{\frac{dp_t(\mathbf{z}_t)}{dt}}_{\text{total derivative}} = \underbrace{\frac{\partial p_t(\mathbf{z}_t)}{\partial t}}_{\text{partial derivative}} + \underbrace{\frac{\partial p_t(\mathbf{z}_t)}{\partial \mathbf{z}_t}}_{\nabla p_t(\mathbf{z}_t)} \cdot \underbrace{\frac{d\mathbf{z}_t}{dt}}_{\text{ODE}}$$

- Total derivative - explicit time dependence of distribution *and* change in state variable
- Partial derivative - keep $\mathbf{z}$ fixed and only consider change of distribution:

$$\frac{\partial p_t(\mathbf{z})}{\partial t} = \lim_{\Delta t \to 0} \frac{p_{t+\Delta t}(\mathbf{z}) - p_t(\mathbf{z})}{\Delta t}$$

ODE for $\log p_t(x_t)$ continued

- Expand right hand of Liouville:

$$\frac{\partial p_t(z_t)}{\partial t} = -\nabla \cdot [\mathbf{f}(\mathbf{z}_t, t) p_t(\mathbf{z}_t)] = -p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) - \mathbf{f}(\mathbf{z}_t, t) \cdot \nabla p_t(\mathbf{z}_t)$$

- Into total derivative expression:

$$\begin{aligned}
\frac{dp_t(\mathbf{z}_t)}{dt} &= \frac{\partial p_t(\mathbf{z}_t)}{\partial t} + \nabla p_t(\mathbf{z}_t) \cdot \frac{d\mathbf{z}_t}{dt} \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) - \mathbf{f}(\mathbf{z}_t, t) \cdot \nabla p_t(\mathbf{z}_t) + \nabla p_t(\mathbf{z}_t) \cdot \frac{d\mathbf{z}_t}{dt} \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) + \underbrace{\left(\frac{d\mathbf{z}_t}{dt} - \mathbf{f}(\mathbf{z}_t, t)\right)}_{=0\,(\mathrm{ODE})} \cdot \nabla p_t(\mathbf{z}_t) \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t)
\end{aligned}$$

ODE for $\log p_t(x_t)$ continued

- Expand right hand of Liouville:

$$\frac{\partial p_t(z_t)}{\partial t} = -\nabla \cdot [\mathbf{f}(\mathbf{z}_t, t) p_t(\mathbf{z}_t)] = -p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) - \mathbf{f}(\mathbf{z}_t, t) \cdot \nabla p_t(\mathbf{z}_t)$$

- Into total derivative expression:

$$\begin{aligned}
\frac{dp_t(\mathbf{z}_t)}{dt} &= \frac{\partial p_t(\mathbf{z}_t)}{\partial t} + \nabla p_t(\mathbf{z}_t) \cdot \frac{d\mathbf{z}_t}{dt} \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) - \mathbf{f}(\mathbf{z}_t, t) \cdot \nabla p_t(\mathbf{z}_t) + \nabla p_t(\mathbf{z}_t) \cdot \frac{d\mathbf{z}_t}{dt} \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t) + \underbrace{\left(\frac{d\mathbf{z}_t}{dt} - \mathbf{f}(\mathbf{z}_t, t)\right)}_{=0\,(\mathrm{ODE})} \cdot \nabla p_t(\mathbf{z}_t) \\
&= p_t(\mathbf{z}_t)\nabla \cdot \mathbf{f}(\mathbf{z}_t, t)
\end{aligned}$$

- Now use $\frac{d}{dt} p_t(\mathbf{z}_t) = p_t(\mathbf{z}_t) \frac{d}{dt} \log p_t(\mathbf{z}_t)$:

$$\frac{d}{dt} \log p_t(\mathbf{z}_t) = -\nabla \cdot \mathbf{f}(\mathbf{z}_t, t)$$

ODE $\log p_1(\mathbf{z}_1)$ and generation recap

- State and log probability ODE:

$$\frac{d}{dt}\begin{bmatrix}\mathbf{z} \\ \log p_t(\mathbf{z})\end{bmatrix} = \begin{bmatrix}\mathbf{f}(\mathbf{z}, t) \\ -\nabla \cdot \mathbf{f}(\mathbf{z}, t)\end{bmatrix}$$

- *Flow map* $\phi_s^t(\mathbf{z}_s)$: Solution to ODE at time t when initialized to $\mathbf{z}_s$ at time s

ODE $\log p_1(\mathbf{z}_1)$ and generation recap

- State and log probability ODE:

$$\frac{d}{dt}\begin{bmatrix}\mathbf{z}\\ \log p_t(\mathbf{z})\end{bmatrix} = \begin{bmatrix}\mathbf{f}(\mathbf{z},t)\\ -\nabla\cdot\mathbf{f}(\mathbf{z},t)\end{bmatrix}$$

- *Flow map* $\phi_s^t(\mathbf{z}_s)$: Solution to ODE at time t when initialized to $\mathbf{z}_s$ at time s
- Log $\log p_1(\mathbf{z}_1)$ - integrate from data: $t=1$ to prior: $t=0$:

$$\log p_1(\mathbf{z}_1) = \log p_0(\phi_1^0(\mathbf{z}_1)) - \int_0^1 \nabla\cdot\mathbf{f}(\phi_1^t(\mathbf{z}_1),t)dt\ .$$

ODE $\log p_1(\mathbf{z}_1)$ and generation recap

- State and log probability ODE:

$$\frac{d}{dt}\begin{bmatrix}\mathbf{z}\\ \log p_t(\mathbf{z})\end{bmatrix} = \begin{bmatrix}\mathbf{f}(\mathbf{z},t)\\ -\nabla\cdot\mathbf{f}(\mathbf{z},t)\end{bmatrix}$$

- *Flow map* $\phi_s^t(\mathbf{z}_s)$: Solution to ODE at time t when initialized to $\mathbf{z}_s$ at time s
- Log $\log p_1(\mathbf{z}_1)$ - integrate from data: $t=1$ to prior: $t=0$:

$$\log p_1(\mathbf{z}_1) = \log p_0(\phi_1^0(\mathbf{z}_1)) - \int_0^1 \nabla\cdot\mathbf{f}(\phi_1^t(\mathbf{z}_1),t)dt\ .$$

- Sample prior, (Euler) integrate and sample data:

$$\begin{aligned}\mathbf{z}_0 &\sim p_0(\mathbf{z}_0)\\ \mathbf{z}_{t+dt} &= \mathbf{z}_t + \mathbf{f}(\mathbf{z}_t,t)dt, \quad \text{for} \quad t = 0, dt, 2dt, \ldots, 1-dt\\ \mathbf{x} &\sim p(\mathbf{x}|\mathbf{z}_1)\end{aligned}$$

- ODE ELBO - downside *simulation-based* - we need to integrate!

$$\log p(\mathbf{x}) \geq \mathcal{L}_{\text{ODE}}(\mathbf{x}) := \mathbb{E}_{q_1(\mathbf{z}_1|\mathbf{x})}\left[\log\frac{p(\mathbf{x}|\mathbf{z}_1)p_1(\mathbf{z}_1)}{q_1(\mathbf{z}_1|\mathbf{x})}\right]$$

ODE pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

ODE pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

Def `ODESolver(`$\mathbf{z}_{\text{init}}$, t_{init}, Δt`):`

$\mathbf{z}, t, \text{logP} = \mathbf{z}_{\text{init}}, t_{\text{init}}, 0$

for *i=1,...,T* **do**

$\mathbf{z} = \mathbf{z} + \mathbf{f}(\mathbf{z}, t)\Delta t$

$\text{logP} = \text{logP} - \nabla \cdot \mathbf{f}(\mathbf{z}, t)\Delta t$

$t = t + \Delta t$

return $\mathbf{z}, \text{logP}$

ODE pseudo code

Input: T (number of discretization steps), $\mathbf{f}$ (drift function), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

Def `ODESolver(`$\mathbf{z}_{\text{init}}$, t_{init}, Δt`):`

  $\mathbf{z}, t, \text{logP} = \mathbf{z}_{\text{init}}, t_{\text{init}}, 0$

  for *i=1,...,T* **do**

    $\mathbf{z} = \mathbf{z} + \mathbf{f}(\mathbf{z}, t)\Delta t$

    $\text{logP} = \text{logP} - \nabla \cdot \mathbf{f}(\mathbf{z}, t)\Delta t$

    $t = t + \Delta t$

  return $\mathbf{z}, \text{logP}$

Def `ODEELBO(`$\mathbf{x}$`):`

  $\mathbf{z}_1 \sim q_1(\mathbf{z}_1|\mathbf{x})$

  $\mathbf{z}_0, \text{ELBO} =$ `ODESolver(`$\mathbf{z}_{\text{init}} = \mathbf{z}_1$, $t_{\text{init}} = 1$, $\Delta t = -1/T$`)`

  $\text{ELBO} = \text{ELBO} + \log p_0(\mathbf{z}_0) + \log p(\mathbf{x}|\mathbf{z}_1) - \log q_1(\mathbf{z}_1|\mathbf{x})$

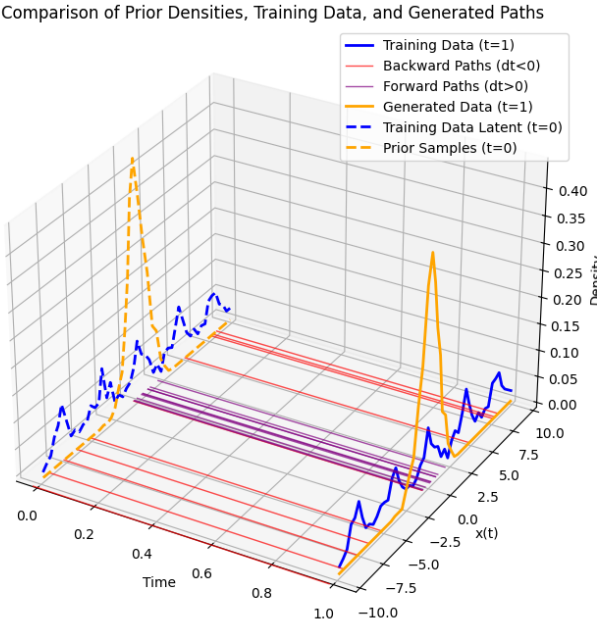
  return ELBO

ODE pseudo code

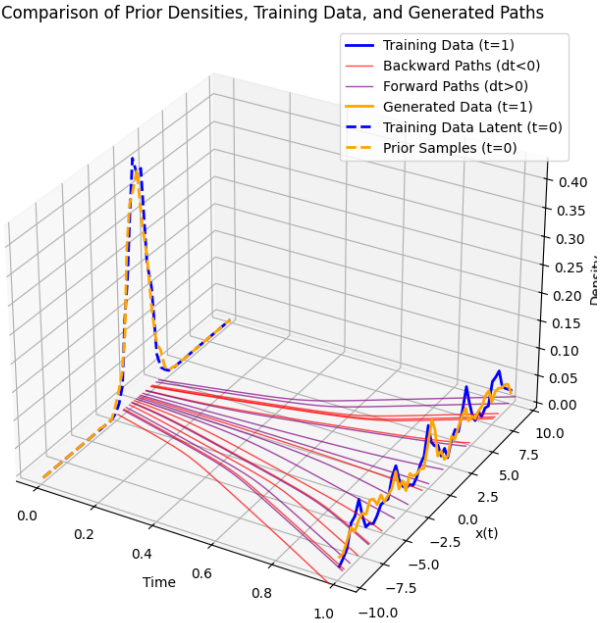
Input: T (number of discretization steps), $\mathbf{f}$ (drift function), p_0 (prior), p (likelihood function), q (variational distribution), $\mathbf{x}$ (data)

```
Def ODESolver(z_init, t_init, Δt):
    z, t, logP = z_init, t_init, 0
    for i=1,...,T do
        z = z + f(z, t)Δt
        logP = logP − ∇ · f(z, t)Δt
        t = t + Δt
    return z, logP
Def ODEELBO(x):
    z_1 ~ q_1(z_1|x)
    z_0, ELBO = ODESolver(z_init = z_1, t_init = 1, Δt = −1/T)
    ELBO = ELBO + log p_0(z_0) + log p(x|z_1) − log q_1(z_1|x)
    return ELBO
Def Sample():
    z_0 ~ p_0(z_0)
    z_1, _ = ODESolver(z_init = z_0, t_init = 0, Δt = 1/T)
    x_sample ~ p(x_sample|z_1)
    return z_sample
```

ODE 1d example at initialization



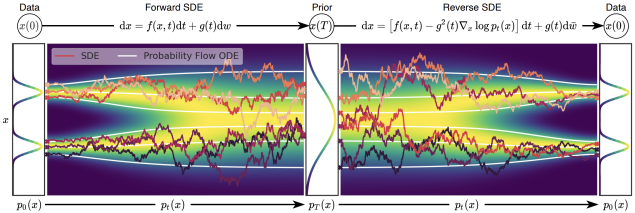
ODE 1d example after training



Part 4.4:
Flow matching

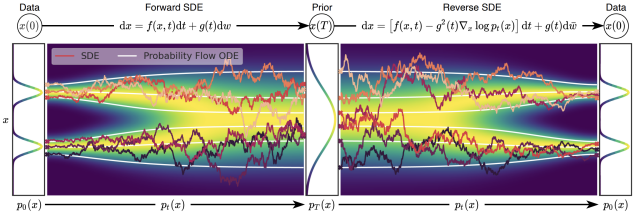
Flow matching - More to know about SDEs part 2...

- Notice the white lines in the plot from Song et al, NeurIPS, 2021



Flow matching - More to know about SDEs part 2...

- Notice the white lines in the plot from Song et al, NeurIPS, 2021



- Forward SDE, *ODE* and reverse-time SDE with the same solution to Fokker-Planck!

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t)dt + \sigma(t)d\overrightarrow{\boldsymbol{W}}_t$$

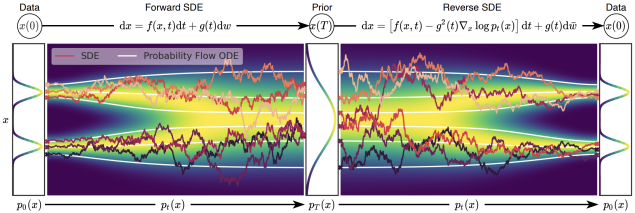
$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{\overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t) - \frac{\sigma^2(t)}{2}\nabla \log q_t(\mathbf{z}_t|\mathbf{x})}_{\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)}\, dt$$

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{\overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t) - \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|\mathbf{x})}_{\overleftarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t)}\, dt + \sigma(t)d\overleftarrow{\boldsymbol{W}}_t$$

- New notation $\overrightarrow{\mathbf{g}}$ for forward SDE drift to distinguish from ODE drift $\mathbf{g}$.

Flow matching - More to know about SDEs part 2...

- Notice the white lines in the plot from Song et al, NeurIPS, 2021



- Forward SDE, *ODE* and reverse-time SDE with the same solution to Fokker-Planck!

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t)dt + \sigma(t)d\overrightarrow{\boldsymbol{W}}_t$$

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{\overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t) - \frac{\sigma^2(t)}{2}\nabla \log q_t(\mathbf{z}_t|\mathbf{x})}_{\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)}\, dt$$

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{\overrightarrow{\mathbf{g}}(\mathbf{x}, \mathbf{z}_t, t) - \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|\mathbf{x})}_{\overleftarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t)}\, dt + \sigma(t)d\overleftarrow{\boldsymbol{W}}_t$$

- New notation $\overrightarrow{\mathbf{g}}$ for forward SDE drift to distinguish from ODE drift $\mathbf{g}$.
- Flow matching use *flow* $\mathbf{g}$ and diffusion loss - *simulation-free* algorithm!

Flow matching

- Remember score matching where we expressed the diffusion loss:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\overrightarrow{\mathbf{f}}(\mathbf{z}_t,t)-\overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- in terms of the score function $\nabla \log q_t(\mathbf{z}_t|\mathbf{x})$.

Flow matching

- Remember score matching where we expressed the diffusion loss:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\overrightarrow{\mathbf{f}}(\mathbf{z}_t,t)-\overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- in terms of the score function $\nabla \log q_t(\mathbf{z}_t|\mathbf{x})$.
- Flow matching: express loss in term of the *flow* = drift in ODE = $\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)$

Flow matching

- Remember score matching where we expressed the diffusion loss:

$$\frac{1}{2}\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\overrightarrow{\mathbf{f}}(\mathbf{z}_t,t)-\overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- in terms of the score function $\nabla \log q_t(\mathbf{z}_t|\mathbf{x})$.
- Flow matching: express loss in term of the *flow* = drift in ODE = $\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)$
- Relation from previous slide:

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t) - \frac{\sigma^2(t)}{2}\nabla \log q_t(\mathbf{z}_t|\mathbf{x})$$
$$\overleftarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t) = \overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t) - \sigma^2(t)\nabla \log q_t(\mathbf{z}_t|\mathbf{x})$$

- Eliminate score function dependence (and again choose $\overleftarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t) = \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t)$):

$$\overrightarrow{\mathbf{g}}(\mathbf{x},\mathbf{z}_t,t) = 2\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) - \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t)$$

- Write generative drift in terms of *flow* (neural network) $\mathbf{v}(\mathbf{z},t)$

$$\overrightarrow{\mathbf{f}}(\mathbf{z}_t,t) = 2\mathbf{v}(\mathbf{z}_t,t) - \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t)$$

- Derivation is done differently in flow matching paper!

Flow matching loss and generation

- Flow matching loss:

$$2\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\mathbf{v}(\mathbf{z}_t,t)-\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- Gaussian Variational distribution used: $q_t(\mathbf{z}_t|\mathbf{x}) = \mathcal{N}(\mathbf{z}_t|t\mathbf{x},(1-t)^2\mathbf{I})$
- Often written as

$$\mathbf{z}_t = t\mathbf{x} + (1-t)\boldsymbol{\epsilon}$$

  with $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0},\mathbf{I})$
- The variational drift follows from q_t (derived in the Bonus material):

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \frac{\mathbf{x}-\mathbf{z}}{1-t}$$

Flow matching loss and generation

- Flow matching loss:

$$2\int_0^1 \mathbb{E}_{q_t(\mathbf{z}_t|\mathbf{x})}\left[\frac{||\mathbf{v}(\mathbf{z}_t,t)-\mathbf{g}(\mathbf{x},\mathbf{z}_t,t)||^2}{\sigma^2(t)}\right]dt$$

- Gaussian Variational distribution used: $q_t(\mathbf{z}_t|\mathbf{x}) = \mathcal{N}(\mathbf{z}_t|t\mathbf{x},(1-t)^2\mathbf{I})$
- Often written as

$$\mathbf{z}_t = t\mathbf{x} + (1-t)\boldsymbol{\epsilon}$$

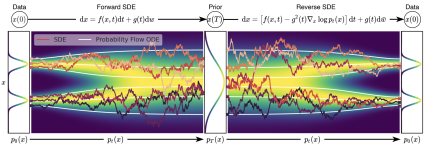
  with $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0},\mathbf{I})$
- The variational drift follows from q_t (derived in the Bonus material):

$$\mathbf{g}(\mathbf{x},\mathbf{z}_t,t) = \frac{\mathbf{x}-\mathbf{z}}{1-t}$$

- Generation - two approaches (ODE and SDE)

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{\mathbf{v}(\mathbf{z}_t,t)}_{\mathbf{f}(\mathbf{x},\mathbf{z}_t,t)}dt$$

$$\mathbf{z}_{t+dt} = \mathbf{z}_t + \underbrace{2\mathbf{v}(\mathbf{z}_t,t) - \overleftarrow{\mathbf{g}}(\mathbf{z}_t,t)}_{\overrightarrow{\mathbf{f}}(\mathbf{x},\mathbf{z}_t,t)}dt + \sigma(t)d\overrightarrow{\boldsymbol{W}}_t$$



- Will be the same generative model if learned exactly.

Part 5:
Wrapping up

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.
- Honestly don't know if this course will put you in a better position than anyone else to take advantage of progress in GenAI

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.
- Honestly don't know if this course will put you in a better position than anyone else to take advantage of progress in GenAI
- The course has covered a lot of ground:
- Methodology -
  - Probabability
  - Algebra
  - Optimization
  - Learning paradigms
  - Architectures (inductive bias)

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.
- Honestly don't know if this course will put you in a better position than anyone else to take advantage of progress in GenAI
- The course has covered a lot of ground:
- Methodology -
  - Probabability
  - Algebra
  - Optimization
  - Learning paradigms
  - Architectures (inductive bias)
- Practice -
  - Python/Pytorch programming
  - MLOps (a little bit)
  - HPC challenges (a little bit)
  - Coding with LLMs
  - Using frameworks (a little bit)

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.
- Honestly don't know if this course will put you in a better position than anyone else to take advantage of progress in GenAI
- The course has covered a lot of ground:
- Methodology -
  - Probabability
  - Algebra
  - Optimization
  - Learning paradigms
  - Architectures (inductive bias)
- Practice -
  - Python/Pytorch programming
  - MLOps (a little bit)
  - HPC challenges (a little bit)
  - Coding with LLMs
  - Using frameworks (a little bit)
- We have *not* covered reinforcement learning and latest reinforcement learning inspired techniques for finetuning large language models.

Why bother about spending time on deep learning

- My sales pitch: If you spend the time then you will become very good at a topic that is changing technology right now.
- Honestly don't know if this course will put you in a better position than anyone else to take advantage of progress in GenAI
- The course has covered a lot of ground:
- Methodology -
  - Probabability
  - Algebra
  - Optimization
  - Learning paradigms
  - Architectures (inductive bias)
- Practice -
  - Python/Pytorch programming
  - MLOps (a little bit)
  - HPC challenges (a little bit)
  - Coding with LLMs
  - Using frameworks (a little bit)
- We have *not* covered reinforcement learning and latest reinforcement learning inspired techniques for finetuning large language models.
- Remember to fill in the evaluation!

Part 6:
Bonus material - derivations

Sketch of derivations

- Mainly for those who want to dig deeper. Not needed for the assignment.
- Fokker-Planck equation
- Forward SDE, ODE and reverse-time SDE with same marginal
- Gaussian marginal distribution and drift function

Sketch of Liouville and Fokker-Planck derivation

- Liouville equation 1d

$$\frac{\partial p_t(z)}{\partial t} = -\frac{\partial}{\partial z}[f(z,t)p_t(z,t)]$$

- Use definition of partial derivative:

$$\frac{\partial p_t(z)}{\partial t} = \lim_{\Delta t \to 0} \frac{p_{t+\Delta t}(z) - p_t(z)}{\Delta t} \ .$$

Sketch of Liouville and Fokker-Planck derivation

- Liouville equation 1d

$$\frac{\partial p_t(z)}{\partial t} = -\frac{\partial}{\partial z}[f(z,t)p_t(z,t)]$$

- Use definition of partial derivative:

$$\frac{\partial p_t(z)}{\partial t} = \lim_{\Delta t \to 0} \frac{p_{t+\Delta t}(z) - p_t(z)}{\Delta t} \ .$$

- Integrate this over arbitrary $h(z)$:

$$\int h(z) \frac{\partial p_t(z)}{\partial t} dx = \lim_{\Delta t \to 0} \frac{1}{\Delta t} \int h(z)(p_{t+\Delta t}(z) - p_t(z)) dz \ .$$

Sketch of Liouville and Fokker-Planck derivation

- Liouville equation 1d

$$\frac{\partial p_t(z)}{\partial t} = -\frac{\partial}{\partial z}[f(z,t)p_t(z,t)]$$

- Use definition of partial derivative:

$$\frac{\partial p_t(z)}{\partial t} = \lim_{\Delta t \to 0} \frac{p_{t+\Delta t}(z) - p_t(z)}{\Delta t} \ .$$

- Integrate this over arbitrary $h(z)$:

$$\int h(z) \frac{\partial p_t(z)}{\partial t} dx = \lim_{\Delta t \to 0} \frac{1}{\Delta t} \int h(z)(p_{t+\Delta t}(z) - p_t(z)) dz \ .$$

- Find $p_{t+\Delta t}(z)$ using Markov property and δ-function:

$$z = z_t + f(z_t,t)\Delta t \quad \rightarrow \quad \delta(z - z_t - f(z_t,t)\Delta t)$$

$$p_{t+\Delta t}(z) = \mathbb{E}_{p_t(z_t)}\left[\delta(z - z_t - f(z_t,t)\Delta t)\right]$$

$$\int h(z) p_{t+\Delta t}(z) dz = \mathbb{E}_{p_t(z_t)}\left[h(z_t + f(z_t,t)\Delta t)\right] \ .$$

Sketch of Liouville and Fokker-Planck derivation cont

- In $\Delta t \to 0$ limit we can expand and only keep first order term:

$$h(z_t + f(x_t, t)\Delta t) = h(z_t) + \frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\Delta t + \mathcal{O}((\Delta t)^2) \ .$$

$$\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z_t)}\left[h(x_t + f(z_t, t)\Delta t)\right] - \mathbb{E}_{p_t(z_t)}\left[h(z_t)\right]$$

$$\to \Delta t\, \mathbb{E}_{p_t(z_t)}\left[\frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\right]$$

Sketch of Liouville and Fokker-Planck derivation cont

- In $\Delta t \to 0$ limit we can expand and only keep first order term:

$$h(z_t + f(x_t, t)\Delta t) = h(z_t) + \frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\Delta t + \mathcal{O}((\Delta t)^2) \ .$$

$$\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z_t)}\left[h(x_t + f(z_t, t)\Delta t)\right] - \mathbb{E}_{p_t(z_t)}\left[h(z_t)\right]$$
$$\to \Delta t\, \mathbb{E}_{p_t(z_t)}\left[\frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\right]$$

- Insert back into orginal expression:

$$\int h(z)\frac{\partial p_t(z)}{\partial t}dz = \lim_{\Delta t \to 0} \frac{1}{\Delta t}\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z)}\left[\frac{\partial h(z)}{\partial z} f(z, t)\right] \ .$$

Sketch of Liouville and Fokker-Planck derivation cont

- In $\Delta t \to 0$ limit we can expand and only keep first order term:

$$h(z_t + f(x_t, t)\Delta t) = h(z_t) + \frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\Delta t + \mathcal{O}((\Delta t)^2) \ .$$

$$\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z_t)}\left[h(x_t + f(z_t, t)\Delta t)\right] - \mathbb{E}_{p_t(z_t)}\left[h(z_t)\right]$$
$$\to \Delta t\, \mathbb{E}_{p_t(z_t)}\left[\frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\right]$$

- Insert back into orginal expression:

$$\int h(z)\frac{\partial p_t(z)}{\partial t}dz = \lim_{\Delta t \to 0} \frac{1}{\Delta t}\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z)}\left[\frac{\partial h(z)}{\partial z} f(z, t)\right] \ .$$

- Integration by parts:

$$\int \frac{\partial h(z)}{\partial z} f(z, t) p_t(z) dz = \underbrace{\left[h(z) f(z, t) p_t(z)\right]_{z=-\infty}^{\infty}}_{=0} - \int h(z)\frac{\partial}{\partial z}\left[f(z, t) p_t(z)\right] dz$$

- Liouville equation must hold because $h(z)$ is arbitrary function.

Sketch of Liouville and Fokker-Planck derivation cont

- In $\Delta t \to 0$ limit we can expand and only keep first order term:

$$h(z_t + f(x_t, t)\Delta t) = h(z_t) + \frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\Delta t + \mathcal{O}((\Delta t)^2) \ .$$

$$\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z_t)}\left[h(x_t + f(z_t, t)\Delta t)\right] - \mathbb{E}_{p_t(z_t)}\left[h(z_t)\right]$$
$$\to \Delta t\, \mathbb{E}_{p_t(z_t)}\left[\frac{\partial h(z_t)}{\partial z_t} f(z_t, t)\right]$$

- Insert back into orginal expression:

$$\int h(z)\frac{\partial p_t(z)}{\partial t}dz = \lim_{\Delta t \to 0} \frac{1}{\Delta t}\int h(z)(p_{t+\Delta t}(z) - p_t(z))dx = \mathbb{E}_{p_t(z)}\left[\frac{\partial h(z)}{\partial z} f(z, t)\right] \ .$$

- Integration by parts:

$$\int \frac{\partial h(z)}{\partial z} f(z, t) p_t(z) dz = \underbrace{\left[h(z) f(z, t) p_t(z)\right]_{z=-\infty}^{\infty}}_{=0} - \int h(z)\frac{\partial}{\partial z}\left[f(z, t) p_t(z)\right] dz$$

- Liouville equation must hold because $h(z)$ is arbitrary function.
- For the stochastic counterpart we can use Itô's lemma to derive Fokker-Planck equation and ODEs, SDE and reverse time SDEs with the same marginal.

Fokker-Planck equation - same marginals

- Fokker-Planck equation

$$\frac{\partial p_t(\mathbf{z})}{\partial t} = -\sum_i \frac{\partial}{\partial z_i}\left[f_i(\mathbf{z},t)p_t(\mathbf{z})\right] + \frac{1}{2}\sum_{i,j}\frac{\partial^2}{\partial z_i \partial z_j}\left[\left[\boldsymbol{\sigma}(\mathbf{z},t)\boldsymbol{\sigma}^T(\mathbf{z},t)\right]_{ij} p_t(\mathbf{z})\right]$$
$$= -\nabla\cdot\left[\mathbf{f}(\mathbf{z},t)p_t(\mathbf{z})\right] + \frac{1}{2}\nabla\cdot\left(\nabla\cdot\left[\boldsymbol{D}(\mathbf{z},t)p_t(\mathbf{z})\right]\right) \ ,$$

  with $\boldsymbol{D}(z,t) = \boldsymbol{\sigma}(\mathbf{z},t)\boldsymbol{\sigma}^T(\mathbf{z},t)$
- Formally we can rewrite the second order term as a first order term:

$$\nabla\cdot\left(\nabla\cdot\left[\boldsymbol{D}(\mathbf{z},t)p_t(\mathbf{z})\right]\right) = \nabla\cdot\left[p_t(\mathbf{z})\underbrace{\frac{1}{p_t(\mathbf{z})}\nabla\cdot\left[\boldsymbol{D}(\mathbf{z},t)p_t(\mathbf{z})\right]}_{\text{(new drift contribution)}}\right]$$

- Rewrite the Fokker-Planck for the ODE (Liouville) to Fokker-Planck for SDE:

$$\frac{\partial p_t(\mathbf{z})}{\partial t} = -\nabla\cdot\left[\mathbf{f}(\mathbf{z},t)\ p_t(\mathbf{z})\right]$$
$$= -\nabla\cdot\left[\left(\mathbf{f}(\mathbf{z},t) + \frac{1}{2\,p_t(\mathbf{z})}\nabla\cdot\left[\boldsymbol{D}(\mathbf{z},t)p_t(\mathbf{z})\right]\right)\ p_t(\mathbf{z})\right] + \frac{1}{2}\nabla\cdot\left(\nabla\cdot\left[\boldsymbol{D}(\mathbf{z},t)p_t(\mathbf{z})\right]\right)$$

- A Fokker-Planck with negative second order term corresponds to the reverse-time process.
- Long derivation to show this. Equivalent path distributions Anderson, 1982.

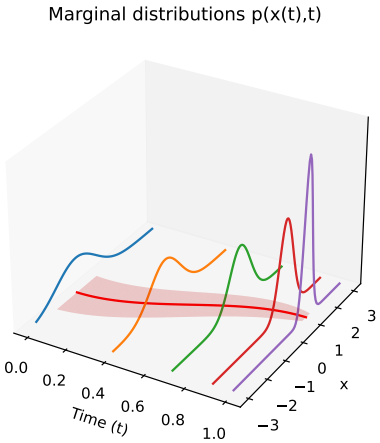
ODE drift for Gaussian marginal

- Task: Find ODE $\frac{d\mathbf{z}}{dt} = \mathbf{g}(\mathbf{x}, \mathbf{z}, t)$ with
- Gaussian variational marginal:

$$q_t(\mathbf{z}|\boldsymbol{\alpha}(\mathbf{x}, t), \beta^2(t)\boldsymbol{I}) \ .$$

- Solution: Define $\mathbf{z}_t = \boldsymbol{\alpha}(\mathbf{x}, t) + \beta(t)\boldsymbol{\epsilon}$.
- With $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$, this will be the right variational
- For each *constant* $\boldsymbol{\epsilon}$, we get a path that has a the ODE we are after!
- To get to ODE, rewrite $\boldsymbol{\epsilon} = \frac{\mathbf{z}_t - \boldsymbol{\alpha}(\mathbf{x}, t)}{\beta(t)}$ and take derivative

$$\frac{d\boldsymbol{\epsilon}}{dt} = \frac{1}{\beta(t)}\frac{d\mathbf{z}_t}{dt} - \frac{1}{\beta(t)}\frac{d\boldsymbol{\alpha}(\mathbf{x}, t)}{dt} - \frac{\mathbf{z}_t - \boldsymbol{\alpha}(\mathbf{x}, t)}{\beta^2(t)}\frac{d\beta(t)}{dt} = 0$$

- Solve with respect to $\frac{d\mathbf{z}_t}{dt} = \mathbf{g}(\mathbf{x}, \mathbf{z}_t, t)$:

$$g(\mathbf{x}, \mathbf{z}_t, t) = \frac{\mathbf{z}_t - \boldsymbol{\alpha}(\mathbf{x}, t)}{\beta(t)}\frac{d\beta(t)}{dt} + \frac{d\boldsymbol{\alpha}(\mathbf{x}, t)}{dt} \ .$$